BITS Pilani

Software Development Practices | Staff Graded Assignment 1

Student Name: UJJWAL JAIN | Student ID: 2024EB02528

System Design Project: FoodExpress Online Food Ordering System

1. Functional and Non-Functional Requirements (4 Marks)

Functional Requirements (FR):

• **FR1 (Restaurant and Menu Browsing):** The system shall enable customers to search, filter by cuisine/location, and view restaurant menus with real-time pricing and availability.

• **FR2 (Cart Management and Checkout):** The system shall allow customers to add items to cart, modify quantities, add special instructions, and submit orders for checkout.

• **FR3 (Payment Processing):** The system shall integrate with external payment gateways to process electronic transactions (Cards, UPI, Net Banking) and issue digital receipts.

• **FR4 (Restaurant Order Management):** The system shall notify restaurants in real time, allowing kitchen staff to accept or reject orders and update preparation status.

• **FR5 (Delivery Assignment and Live Tracking):** The system shall assign confirmed orders to nearby delivery partners and provide customers with live order tracking.

Non-Functional Requirements (NFR):

• **NFR1 (Performance and Latency):** Menu search and checkout processing shall return responses within 1.5 seconds at the 95th percentile under normal operating load.

• **NFR2 (Availability and Uptime):** The core ordering platform shall maintain 99.9% uptime, especially during peak meal rush hours (12:00 to 14:30 and 19:00 to 22:30).

• **NFR3 (Security and Data Protection):** Payment transactions must comply with PCI-DSS Level 1 standards. Customer personal data, passwords, and tokens must be encrypted using AES-256 and bcrypt, with TLS 1.3 enforced.

• **NFR4 (Scalability and Concurrency):** The backend architecture shall scale horizontally to support at least 10,000 concurrent shopping carts and active checkout sessions without transaction deadlocks.

2. Use Case: Place a Food Order (3 Marks)

Use Case Name: Place a Food Order

Scope: FoodExpress Online Ordering System | **Level:** User Goal

Primary Actor: Customer

Secondary Actors: Payment Gateway, Restaurant, Delivery Partner, Notification Service

Stakeholders and Interests: Customer wants fast checkout and accurate delivery ETA. Restaurant wants verified payments before cooking. Delivery Partner wants prompt notification of orders ready for pickup. FoodExpress Platform wants verified transactions and low dispute rates.

Preconditions: (1) Customer is logged in with a verified delivery address. (2) Cart contains at least one item from an active, open restaurant.

Success Postconditions: Order record is persisted in Confirmed state. Payment is authorized. Kitchen ticket is dispatched to Restaurant. Order confirmation with ETA is sent to Customer.

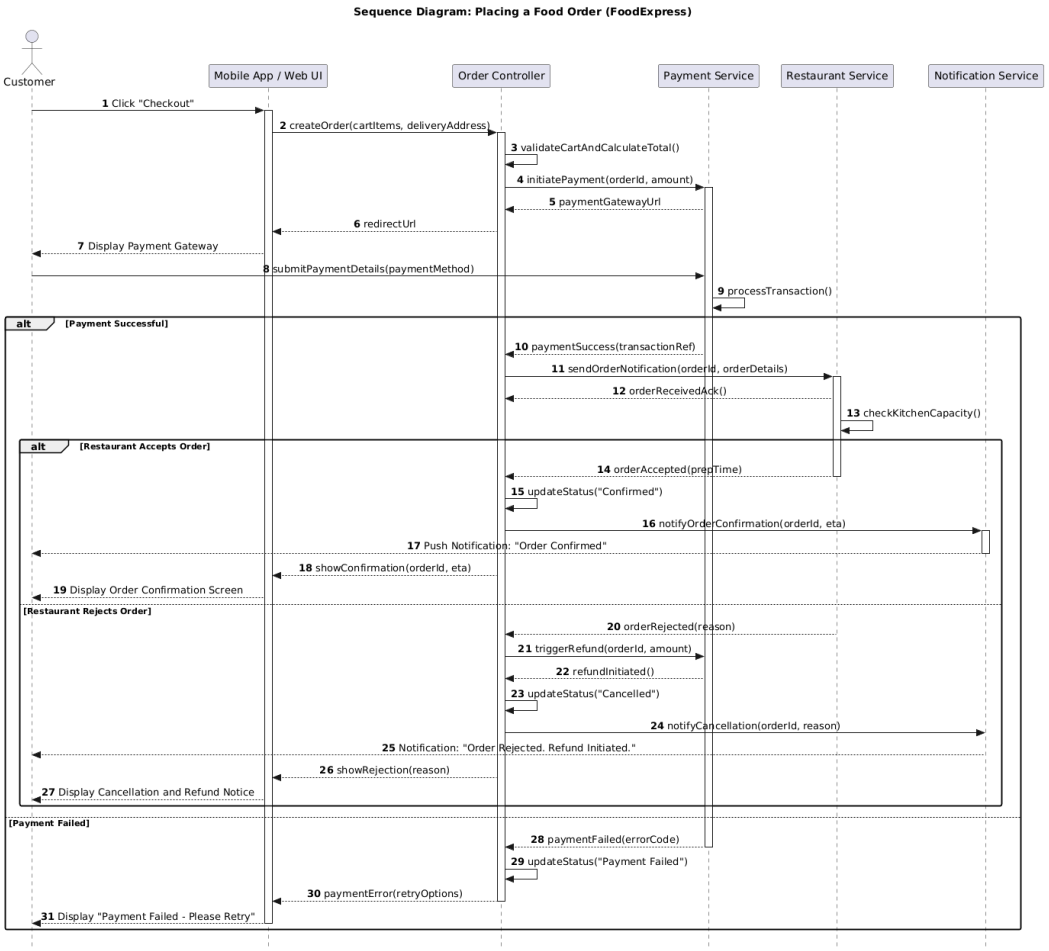
Failure Postconditions: No financial debit occurs (or immediate refund is issued). Customer cart items remain preserved, and clear error reason is shown.

Main Success Scenario (Nominal Flow):
1. Customer navigates to checkout screen and reviews cart items, delivery address, and pricing.
2. Customer selects payment method and clicks 'Place Order'.
3. System generates an order with status 'Payment Pending' and redirects to Payment Gateway.
4. Customer enters payment credentials and confirms transaction.
5. Payment Gateway authorizes transaction and returns payment success token to System.
6. System updates order payment status to 'Payment Received' and forwards order to Restaurant.
7. Restaurant reviews order and clicks 'Accept Order'.
8. System updates order status to 'Confirmed' and sets estimated preparation time.
9. System dispatches confirmation notification with Order ID and ETA to Customer.
10. System registers order in delivery dispatch pool for partner assignment.

Extensions / Alternative Flows:
3a. Item Out of Stock: System alerts Customer, recalculates total, and prompts to modify cart.
5a. Payment Failed: Payment Gateway declines; System displays error message and prompts retry.
7a. Restaurant Rejects Order: Restaurant indicates capacity full; System cancels order, triggers full refund, and notifies Customer.

3. Sequence Diagram: Placing a Food Order (3 Marks)

The sequence diagram illustrates interactions across Customer, Mobile App UI, Order Controller, Payment Service, Restaurant Service, and Notification Service.



PlantUML Source Code:

```
@startuml
skinparam style strictuml
skinparam sequenceMessageAlign center
skinparam roundcorner 5
skinparam BoxPadding 10

title Sequence Diagram: Placing a Food Order (FoodExpress)

actor Customer as cust
participant "Mobile App / Web UI" as ui
participant "Order Controller" as ordCtrl
participant "Payment Service" as paySvc
participant "Restaurant Service" as restSvc
participant "Notification Service" as notifSvc

autonumber

cust -> ui : Click "Checkout"
activate ui
ui -> ordCtrl : createOrder(cartItems, deliveryAddress)
activate ordCtrl

ordCtrl -> ordCtrl : validateCartAndCalculateTotal()
ordCtrl -> paySvc : initiatePayment(orderId, amount)
activate paySvc
paySvc --> ordCtrl : paymentGatewayUrl
ordCtrl --> ui : redirectUrl
ui --> cust : Display Payment Gateway

cust -> paySvc : submitPaymentDetails(paymentMethod)
```

```
paySvc -> paySvc : processTransaction()

alt Payment Successful
paySvc --> ordCtrl : paymentSuccess(transactionRef)
ordCtrl -> restSvc : sendOrderNotification(orderId, orderDetails)
activate restSvc
restSvc --> ordCtrl : orderReceivedAck()

restSvc -> restSvc : checkKitchenCapacity()

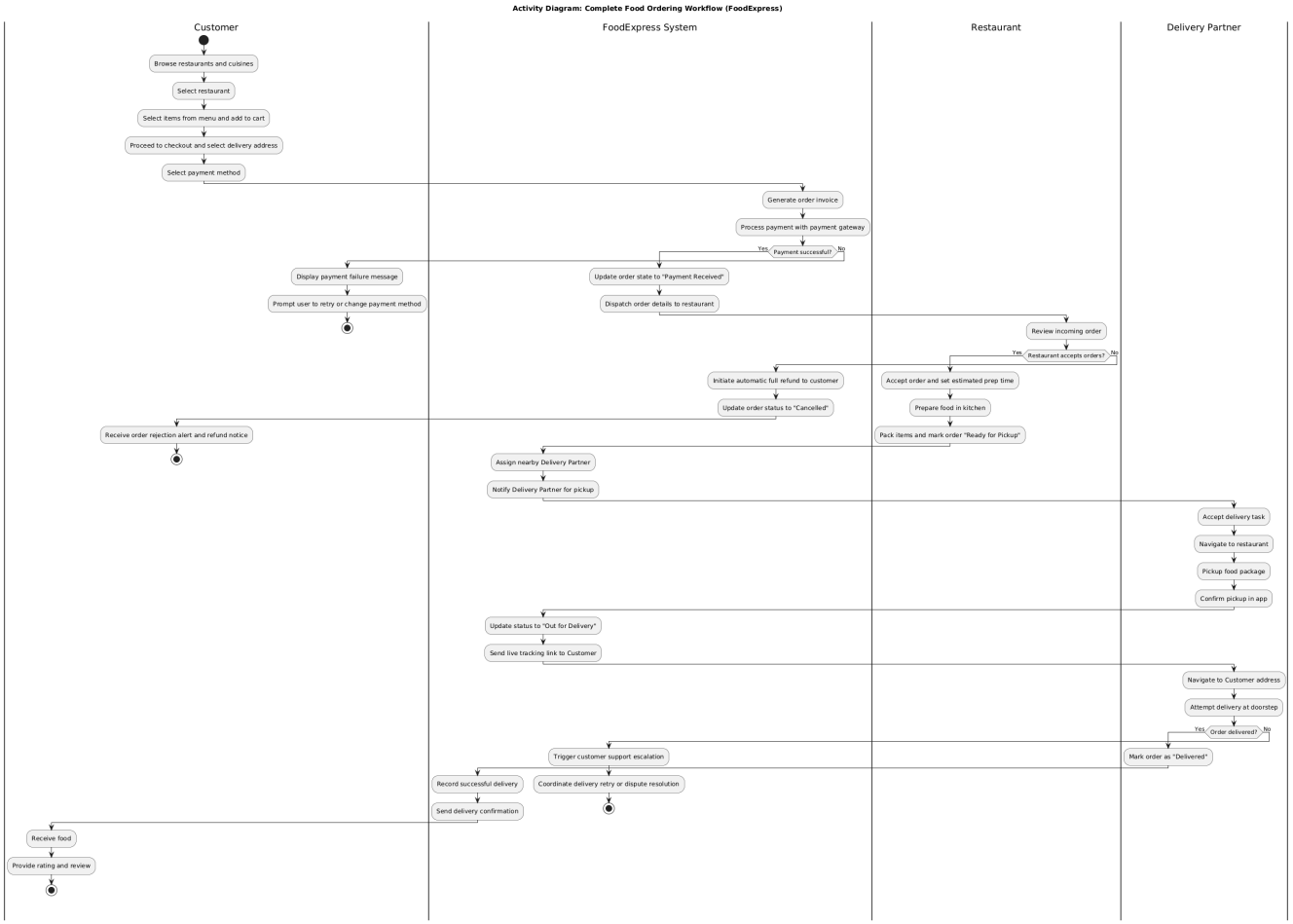
alt Restaurant Accepts Order
restSvc --> ordCtrl : orderAccepted(prepTime)
deactivate restSvc
ordCtrl -> ordCtrl : updateStatus("Confirmed")
ordCtrl -> notifSvc : notifyOrderConfirmation(orderId, eta)
activate notifSvc
notifSvc --> cust : Push Notification: "Order Confirmed"
deactivate notifSvc
ordCtrl --> ui : showConfirmation(orderId, eta)
ui --> cust : Display Order Confirmation Screen
else Restaurant Rejects Order
restSvc --> ordCtrl : orderRejected(reason)
ordCtrl -> paySvc : triggerRefund(orderId, amount)
paySvc --> ordCtrl : refundInitiated()
ordCtrl -> ordCtrl : updateStatus("Cancelled")
ordCtrl -> notifSvc : notifyCancellation(orderId, reason)
notifSvc --> cust : Notification: "Order Rejected. Refund Initiated."
ordCtrl --> ui : showRejection(reason)
ui --> cust : Display Cancellation and Refund Notice
end

else Payment Failed
paySvc --> ordCtrl : paymentFailed(errorCode)
deactivate paySvc
ordCtrl -> ordCtrl : updateStatus("Payment Failed")
ordCtrl --> ui : paymentError(retryOptions)
deactivate ordCtrl
ui --> cust : Display "Payment Failed - Please Retry"
deactivate ui
end

@enduml
```

4. Activity Diagram: Complete Food Ordering Workflow (2 Marks)

The activity diagram models the end-to-end food ordering workflow across Customer, FoodExpress System, Restaurant, and Delivery Partner, highlighting key decision points.



PlantUML Source Code:

```
@startuml
skinparam activityFontSize 12
skinparam activityDiamondFontSize 11
skinparam defaultTextAlignment center

title Activity Diagram: Complete Food Ordering Workflow (FoodExpress)

|Customer|
start
:Browse restaurants and cuisines;
:Select restaurant;
:Select items from menu and add to cart;
:Proceed to checkout and select delivery address;
:Select payment method;

|FoodExpress System|
:Generate order invoice;
:Process payment with payment gateway;

if (Payment successful?) then (Yes)
:Update order state to "Payment Received";
:Dispatch order details to restaurant;

|Restaurant|
:Review incoming order;
if (Restaurant accepts orders?) then (Yes)
:Accept order and set estimated prep time;
:Prepare food in kitchen;
:Pack items and mark order "Ready for Pickup";
```

```
|FoodExpress System|
:Assign nearby Delivery Partner;
:Notify Delivery Partner for pickup;

|Delivery Partner|
:Accept delivery task;
:Navigate to restaurant;
:Pickup food package;
:Confirm pickup in app;

|FoodExpress System|
:Update status to "Out for Delivery";
:Send live tracking link to Customer;

|Delivery Partner|
:Navigate to Customer address;
:Attempt delivery at doorstep;

if (Order delivered?) then (Yes)
:Mark order as "Delivered";

|FoodExpress System|
:Record successful delivery;
:Send delivery confirmation;

|Customer|
:Receive food;
:Provide rating and review;
stop
else (No)
|FoodExpress System|
:Trigger customer support escalation;
:Coordinate delivery retry or dispute resolution;
stop
endif

else (No)
|FoodExpress System|
:Initiate automatic full refund to customer;
:Update order status to "Cancelled";

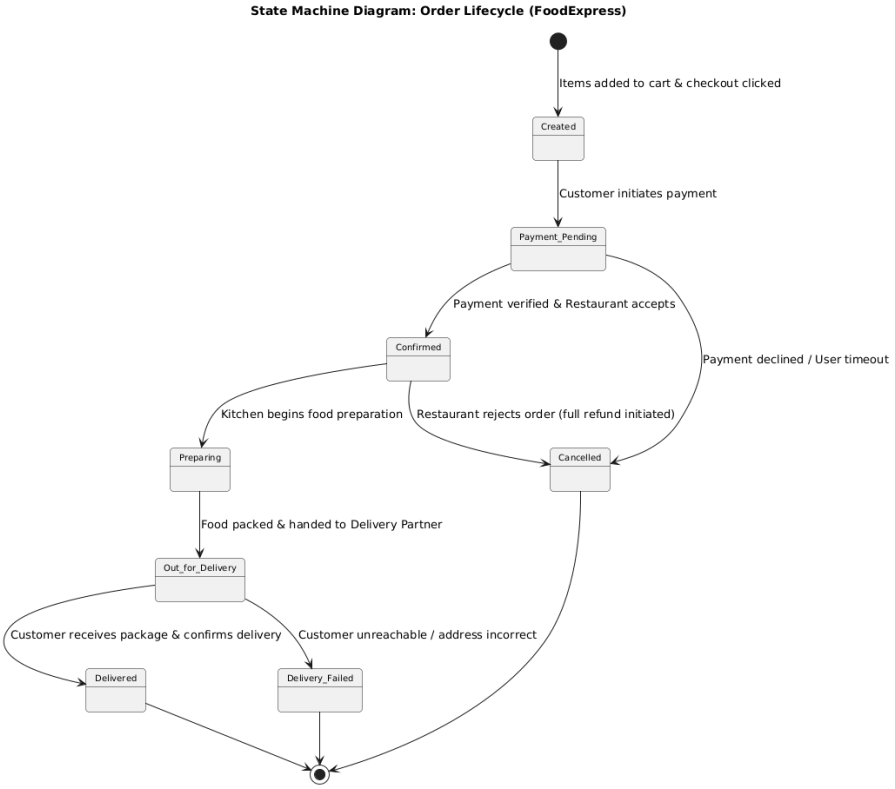
|Customer|
:Receive order rejection alert and refund notice;
stop
endif

else (No)
|Customer|
:Display payment failure message;
:Prompt user to retry or change payment method;
stop
endif

@enduml
```

5. State Diagram: Order Lifecycle (3 Marks)

The state machine diagram illustrates all valid states and transitions for an Order object from creation to delivery or cancellation.



PlantUML Source Code:

```
@startuml
skinparam stateFontSize 12
skinparam stateAttributeFontSize 10
skinparam roundCorner 8

title State Machine Diagram: Order Lifecycle (FoodExpress)

[*] --> Created : Items added to cart & checkout clicked

Created --> Payment_Pending : Customer initiates payment

Payment_Pending --> Confirmed : Payment verified & Restaurant accepts
Payment_Pending --> Cancelled : Payment declined / User timeout

Confirmed --> Preparing : Kitchen begins food preparation
Confirmed --> Cancelled : Restaurant rejects order (full refund initiated)

Preparing --> Out_for_Delivery : Food packed & handed to Delivery Partner

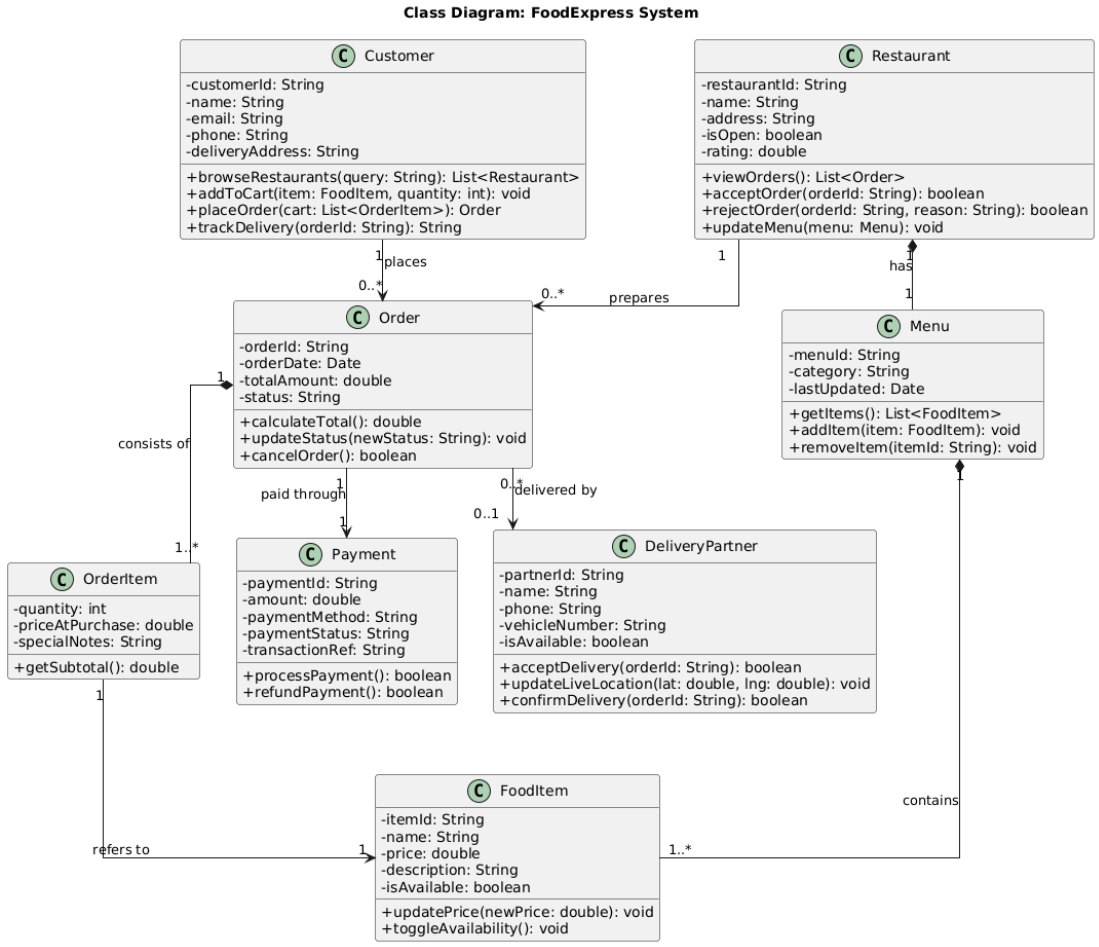
Out_for_Delivery --> Delivered : Customer receives package & confirms delivery
Out_for_Delivery --> Delivery_Failed : Customer unreachable / address incorrect

Delivered --> [*]
Cancelled --> [*]
Delivery_Failed --> [*]

@enduml
```

6. Class Diagram: FoodExpress System (5 Marks)

The class diagram defines the structural domain model including Customer, Restaurant, Menu, FoodItem, Order, OrderItem, Payment, and DeliveryPartner with explicit access modifiers, attributes, methods, and relationships.



PlantUML Source Code:

```
@startuml
skinparam classAttributeIconSize 0
skinparam roundCorner 6
skinparam linetype ortho

title Class Diagram: FoodExpress System

class Customer {
- customerId: String
- name: String
- email: String
- phone: String
- deliveryAddress: String
+ browseRestaurants(query: String): List<Restaurant>
+ addToCart(item: FoodItem, quantity: int): void
+ placeOrder(cart: List<OrderItem>): Order
+ trackDelivery(orderId: String): String
}

class Restaurant {
- restaurantId: String
- name: String
- address: String
- isOpen: boolean
- rating: double
+ viewOrders(): List<Order>
+ acceptOrder(orderId: String): boolean
+ rejectOrder(orderId: String, reason: String): boolean
+ updateMenu(menu: Menu): void
}
```

```
class Menu {
- menuId: String
- category: String
- lastUpdated: Date
+ getItems(): List<FoodItem>
+ addItem(item: FoodItem): void
+ removeItem(itemId: String): void
}

class FoodItem {
- itemId: String
- name: String
- price: double
- description: String
- isAvailable: boolean
+ updatePrice(newPrice: double): void
+ toggleAvailability(): void
}

class Order {
- orderId: String
- orderDate: Date
- totalAmount: double
- status: String
+ calculateTotal(): double
+ updateStatus(newStatus: String): void
+ cancelOrder(): boolean
}

class OrderItem {
- quantity: int
- priceAtPurchase: double
- specialNotes: String
+ getSubtotal(): double
}

class Payment {
- paymentId: String
- amount: double
- paymentMethod: String
- paymentStatus: String
- transactionRef: String
+ processPayment(): boolean
+ refundPayment(): boolean
}

class DeliveryPartner {
- partnerId: String
- name: String
- phone: String
- vehicleNumber: String
- isAvailable: boolean
+ acceptDelivery(orderId: String): boolean
+ updateLiveLocation(lat: double, lng: double): void
+ confirmDelivery(orderId: String): boolean
}

Customer "1" --> "0..*" Order : places
Restaurant "1" *-- "1" Menu : has
Menu "1" *-- "1..*" FoodItem : contains
Order "1" *-- "1..*" OrderItem : consists of
OrderItem "1" --> "1" FoodItem : refers to
Order "1" --> "1" Payment : paid through
Restaurant "1" --> "0..*" Order : prepares
Order "0..*" --> "0..1" DeliveryPartner : delivered by

@enduml
```